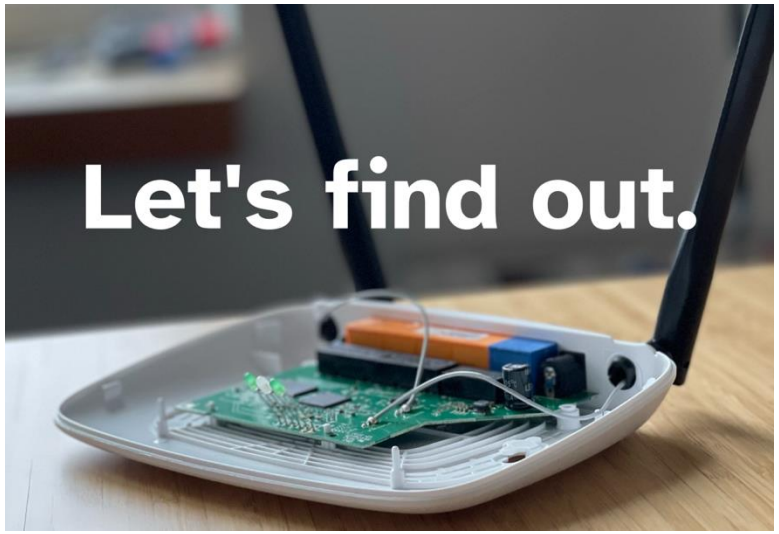
Reliability Solutions + Rowhammer Mitigations

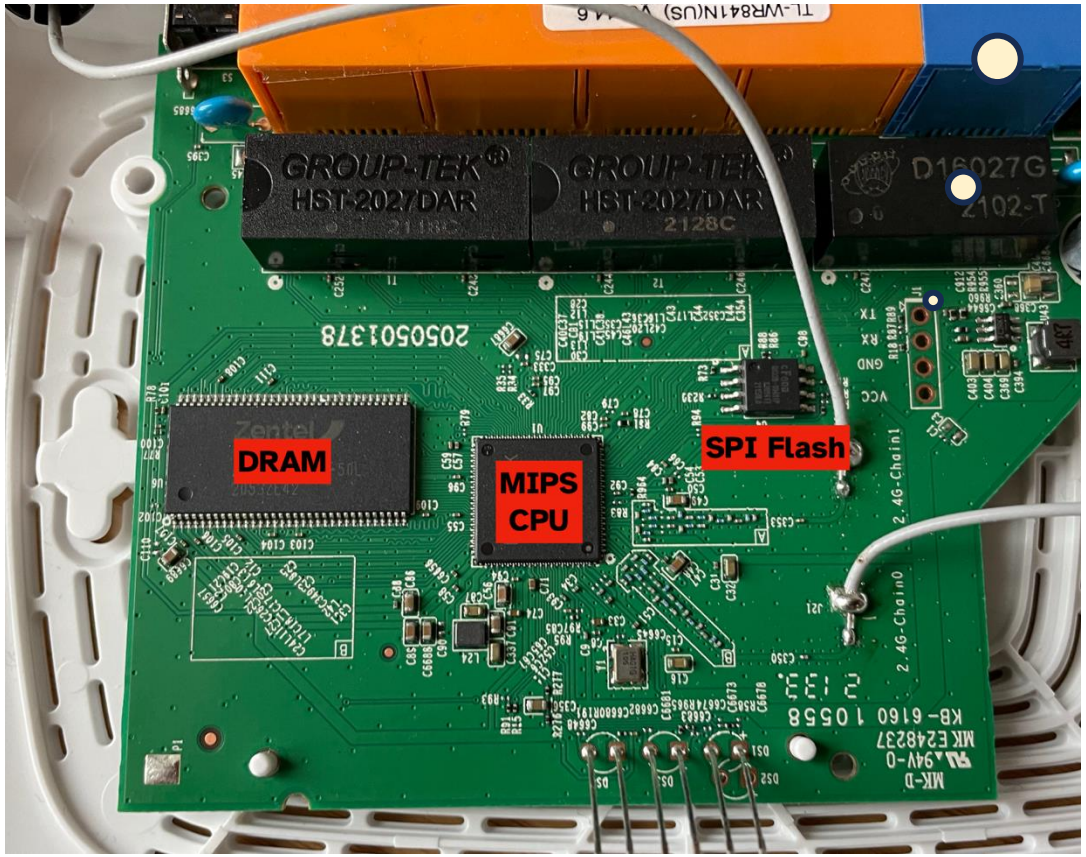
Mengjia Yan

Spring 2026

Leaving Debug Port Open

Why is the debug port so easily accessible?





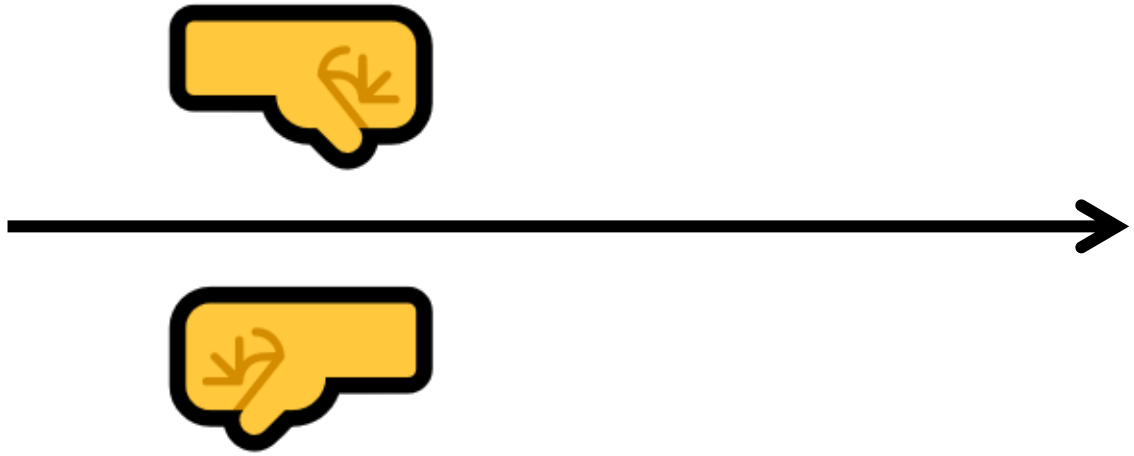
Mitigation Design Considerations

Cost?

Security?

Usability?

Performance?



Outline

- Extreme physical defenses: when chips wear armor
- Engineering for Glitches: Practical Fault Defense
- Taming Disturbance: The Evolution of RowHammer Mitigation

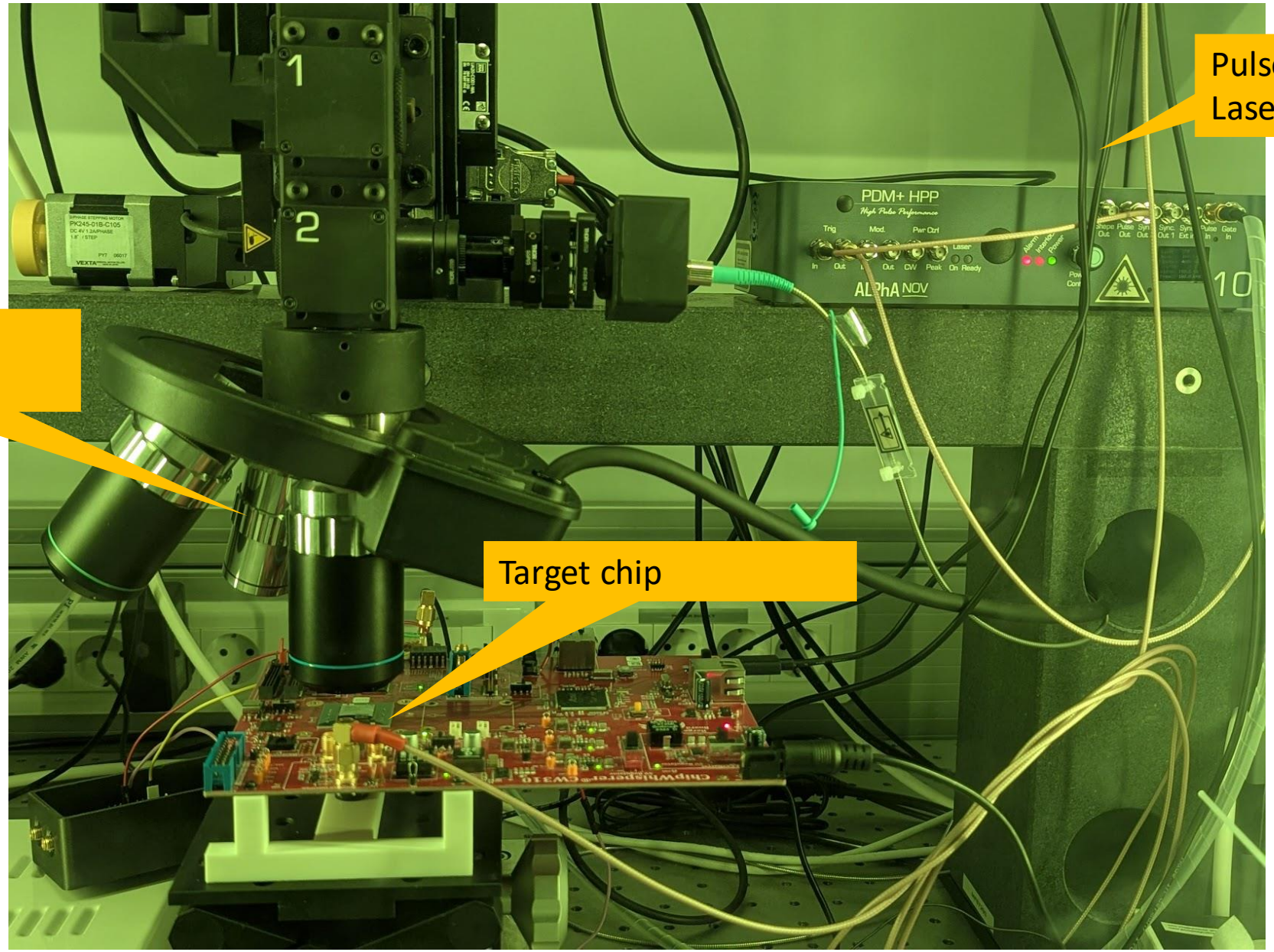
Physical Attack Mitigation Case Study

- IBM 4758
- Satisfy FIPS 140-1 Level 4

1.4 Security Level 4

Security Level 4 provides the highest level of security. Although most existing products do not meet this level of security, some products are commercially available which meet many of the Level 4 requirements. Level 4 physical security provides an envelope of protection around the cryptographic module. Whereas the tamper detection circuits of lower level modules may be bypassed, the intent of Level 4 protection is to detect a penetration of the device from any direction. For example, if one attempts to cut through the enclosure of the cryptographic module, the attempt should be detected and all critical security parameters should be zeroized. Level 4 devices are particularly useful for operation in a physically unprotected environment where an intruder could possibly tamper with the device.

Photo of IBM 4758 Cryptographic Coprocessor (courtesy of Steve Weingart) from *https://www.cl.cam.ac.uk/~rnc1/descrack/ibm4758.html*


Pulse On Demand
Laser Generator
Microscope to find
right position
Target chip
PDM+ HPP
ALPhA NOV

Physical Tamper Resistance

- Make it difficult for the attackers to get access to PCB

Tampering Detection

Make enclosure very robust.
Upon opening lid, disconnect power supply

↓

Drill through the lid

↓

Photocells and tilt devices

Tampering Evident

“Potting" the device in a block of epoxy resin

↓

Patience: Scrape away the epoxy

↓

Tamper-sensing barriers:
nichrome wire wound around the device

IBM 4758 Secure Co-Processor

- Extreme physical security
  - Clock Glitching: Use phase locked loops and independently generated internal clocks
  - Voltage Glitching: Add detection and monitor circuits to watch for voltage changes
  - X-ray fault injection: A radiation sensor
  - Electromagnetic side channels: Solid aluminium shielding and a low-pass filter (a Faraday cage)

Expensive. Other secure processors only focus on a limited set of physical attacks.

- But broken by software-level logic flaw
  - There is a flaw in key management, allowing an attacker to chain multiple legal API calls to leak secrets

https://www.cl.cam.ac.uk/~rnc1/descrack/ibm4758.html

Security Analysis of a Cryptographically-Enabled PIN Processing System IEEE Security & Privacy, 2002.

Mitigation Design Considerations

Cost?

Security?

Usability?

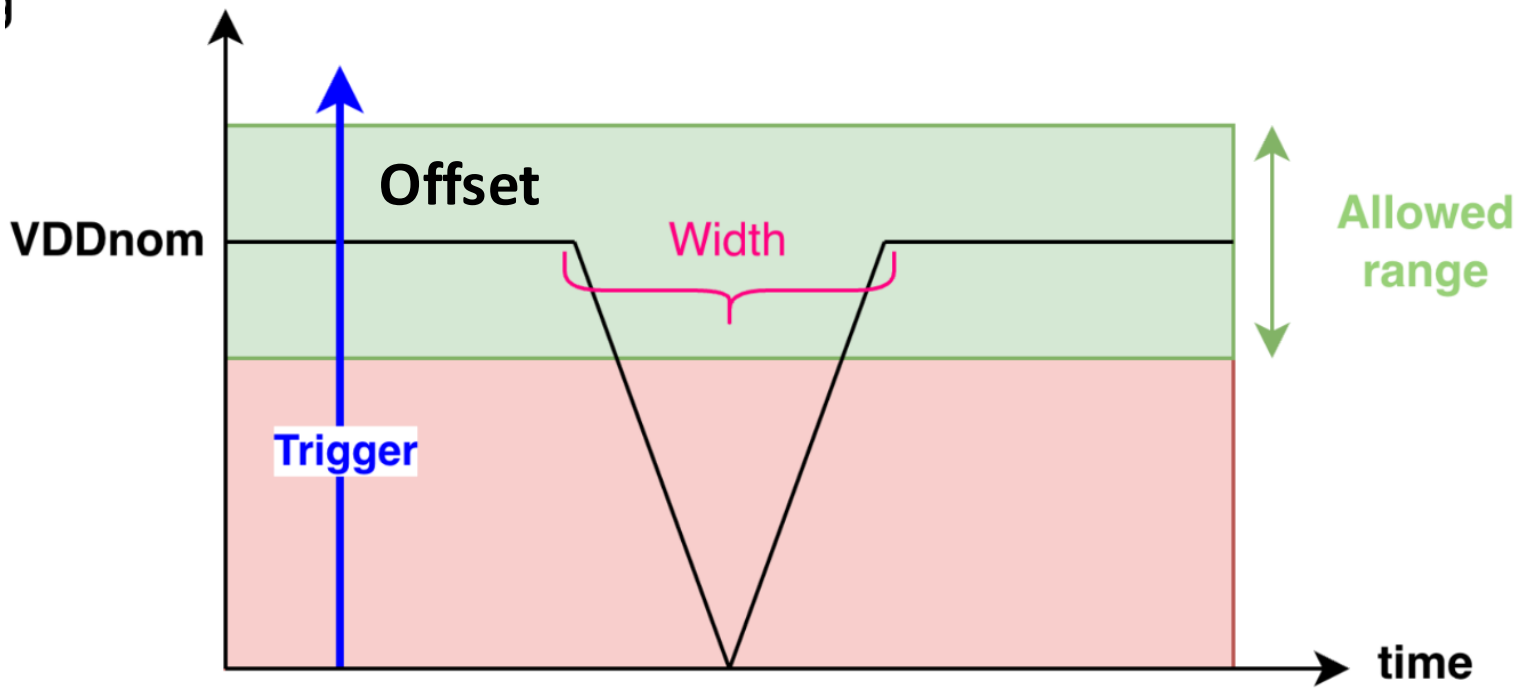
Performance?

Fault Injection Mitigation

Recap: Voltage glitching requires finding the right parameters

Important parameters for voltage glitching

- **Offset** (When after the trigger it should occur)
- **width** of the glitch
- **number** of glitches



Theory: Make Fault Injection Difficult

- Attacker's challenges:
  - Having control over the timing and the location of the fault

- What can be the high-level attack strategies?
  - Approach 1: randomization to make the control more difficult
  - Approach 2: detect anomaly behaviors of the system and block it (e.g., ECC)

Slides credit:

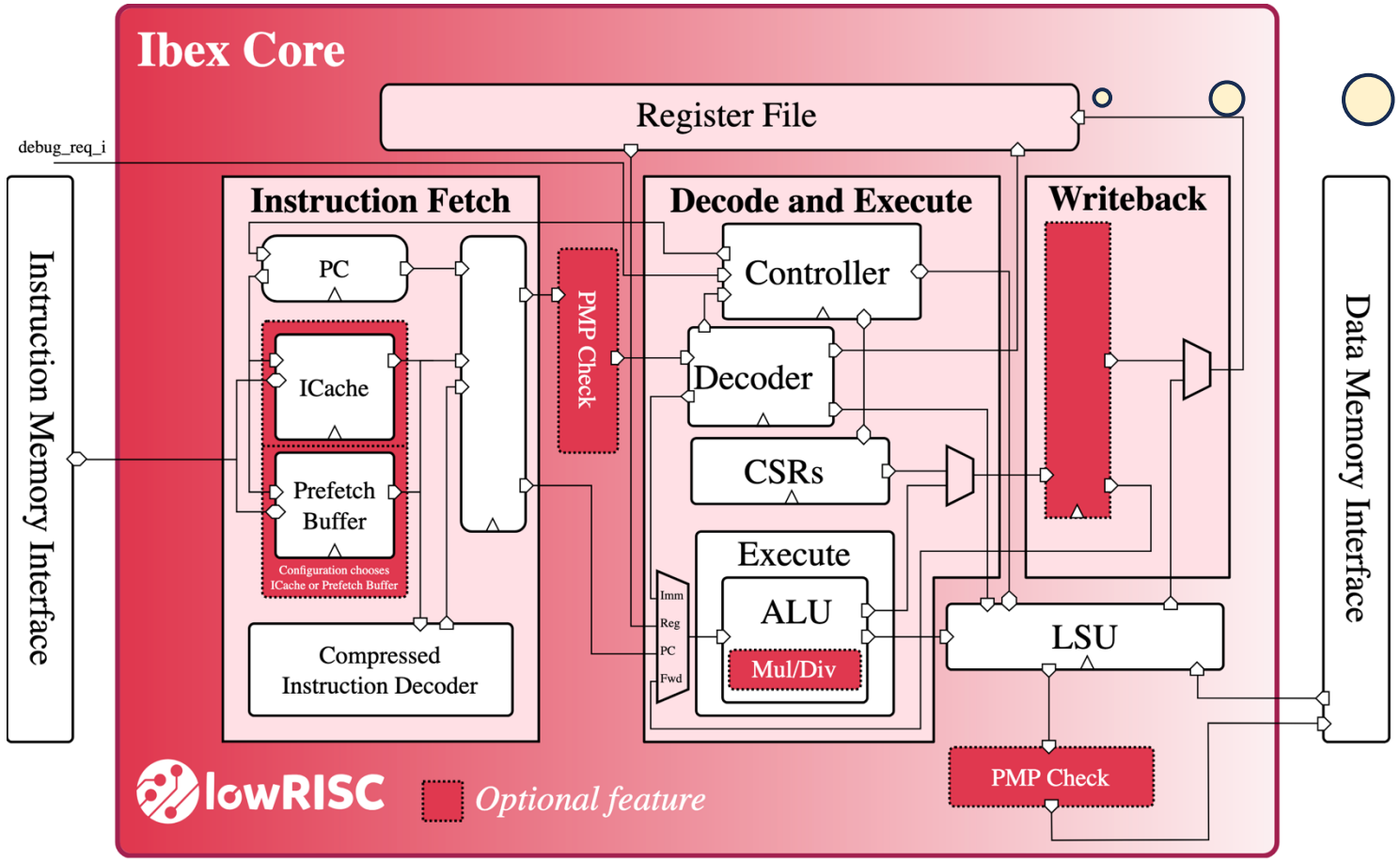
Miles Dai <milesdai@zerorisc.com>
zeroRISC Inc.

Arun Thomas
zeroRISC Inc., VP Engineering

Dominic Rizzo
zeroRISC Inc., CEO
OpenTitan Project Director

Software-based Approaches:
A collection of lessons from OpenTitan Projects

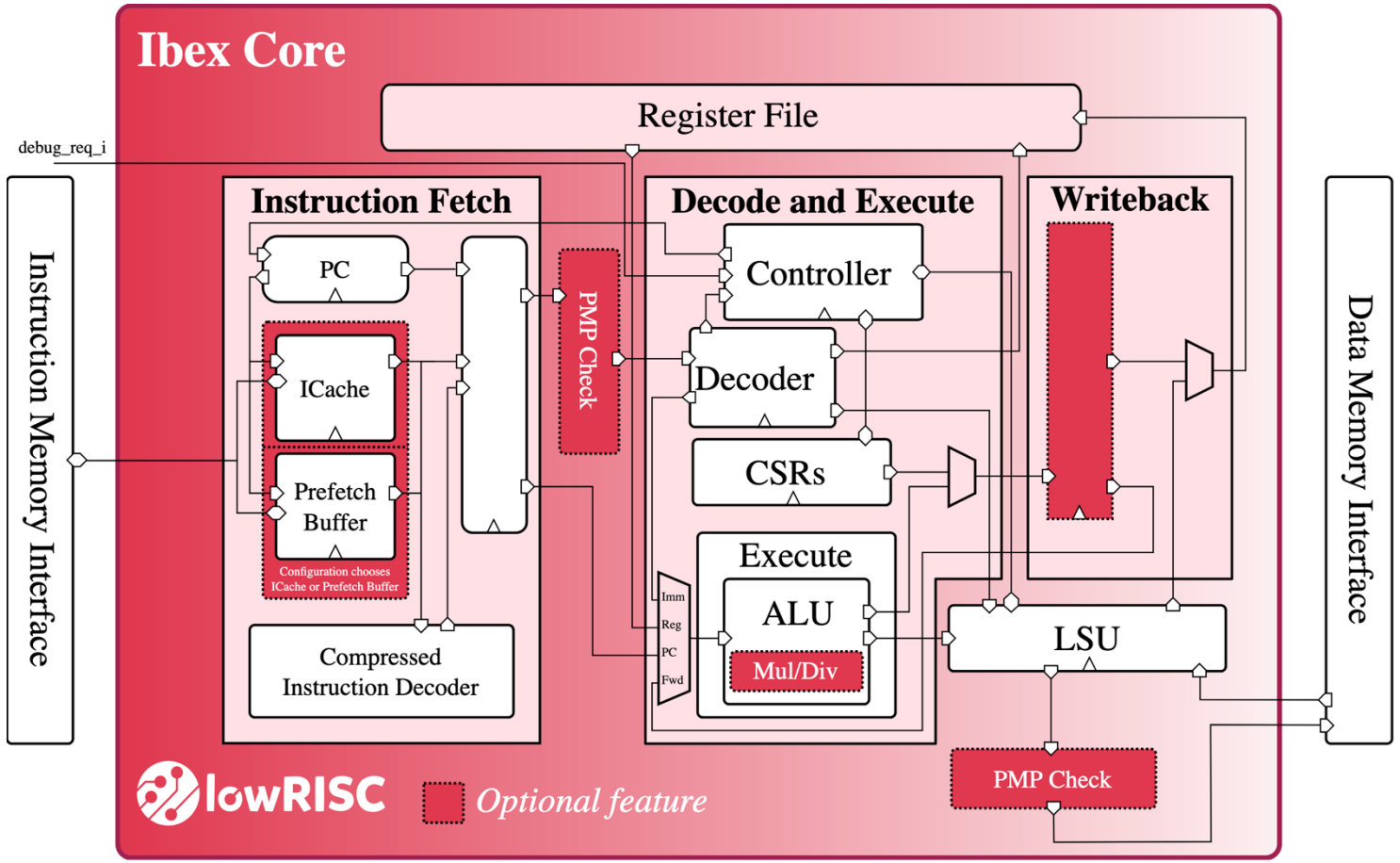
Hardware Assisted Fault Detection



How can we protect ourselves against bit-flips in the register file?

- We can add an ECC to the register file to detect when a value has changed.
- What are the costs associated with ECC?

Challenge: When should we check ECC?



- We can't verify the entire contents of the ECC-protected register file every cycle…
- **Idea:** Verify the ECC immediately before reading a register.
- **Challenge:** Pipeline signaling is complicated, and a "register read" doesn't always come from the register file!
- **CVE-2024-57037 (discovered by Vincent and Peter etc.):** Pipeline forwarding signal is misread, leading to ECC checks being skipped.

What can feasibly be done via fault injection?

The OpenTitan team has identified that some attacks are easier to perform than others!

- Easy
  - Skip one instruction
  - Glitch a register to all 0’s or 1’s
- Hard
  - Set a register to a specific value
  - Multiple precisely-timed glitches
  - Skipping a precise number of instructions

Example 1: Multi-bit Encodings

Make the attacker's life more difficult:

Instead of requiring the attacker to glitch a register to all 0's or 1's, force them to set a register to a specific value…

Multi-bit (MUBI) Encodings

```
enum lifecycle_state {
 // Unlocked test state with debug functions.
 kLcStateTest,

 // Production life cycle state.
 kLcStateProd,

 // RMA life cycle state.
 kLcStateRma,
};
```

```
enum lifecycle_state {
 // Unlocked test state with debug functions.
 kLcStateTest = 0xb2865fbb,

 // Production life cycle state.
 kLcStateProd = 0x65f2520f,

 // RMA life cycle state.
 kLcStateRma = 0xcf8cfaab,
};
```

What is the **minimum** number of bits an attacker needs to flip?

What integers do we use under the hood?

Multi-bit (MUBI) Encodings

```c
/**
* Lifecycle states.
*
* This is a condensed version of the 24 possible life cycle states where
* TEST_UNLOCKED_* states are mapped to `kLcStateTest` and invalid states where
* CPU execution is disabled are omitted.
*
* Encoding generated with
* $ ./util/design/sparse-fsm-encode.py -d 6 -m 5 -n 32 \
*     -s 2447090565 --language=c
*
* Minimum Hamming distance: 13
* Maximum Hamming distance: 19
* Minimum Hamming weight: 15
* Maximum Hamming weight: 20
*/
typedef enum lifecycle_state {
 /**
  * Unlocked test state where debug functions are enabled.
  */
 kLcStateTest = 0xb2865fbb,
 /**
  * Development life cycle state where limited debug functionality is
  * available.
  */
 kLcStateDev = 0x0b5a75e0,
 …
} lifecycle_state_t;
```

lifecycle.h

Example 2: Hash Checking

Passcode or Hash Checking

```
typedef struct hmac_digest {
 uint32_t digest[8];
} hmac_digest_t;

typedef struct boot_data {
  hmac_digest_t digest;  // SHA-256 digest of boot data.
  uint32_t min_security_version_rom_ext;
  uint32_t min_security_version_bl0;
} boot_data_t;
```

How to ensure 8 words in the digest are checked?

Passcode or Hash Checking

```
static const uint32_t shares[8] = {
   0xe021e1a9, 0xf81e8365, 0xbf8322db, 0xc7a37080,
   0xdd8ce33f, 0x7585d574, 0x951777af, 0x271a933f,
};

const uint32_t kErrorOk = shares[0] ^ shares[1] …… ^ shares[7];

bool check_digest(const boot_data_t *boot_data) {
 rom_error_t error = 0;
 hmac_digest_t act_digest;
 boot_data_digest_compute(boot_data, &act_digest);


 for (size_t i = 0; i < 8; ++i) {
   error ^= boot_data->digest[i] ^ act_digest[i] ^ shares[i];
 }
 return error == kErrorOk;
}
```

Pre-compute shares and XORed results

Compute the digest

Generate the valid error value from the shares.

Hash Checking

Any additional hardening opportunities?

```
static const uint32_t shares[8] = {
   0xe021e1a9, 0xf81e8365, 0xbf8322db, 0xc7a37080,
   0xdd8ce33f, 0x7585d574, 0x951777af, 0x271a933f,
};

const uint32_t kErrorOk = shares[0] ^ shares[1] …… ^ shares[7];

bool check_digest(const boot_data_t *boot_data) {
 rom_error_t error = 0;
 hmac_digest_t act_digest;
 boot_data_digest_compute(boot_data, &act_digest);


 for (size_t i = 0; i < 8; ++i) {
   error ^= boot_data->digest[i] ^ act_digest[i] ^ shares[i];
 }
 return error == kErrorOk;
}
```

Example 3: Redundant Condition Checks

Force the attacker:

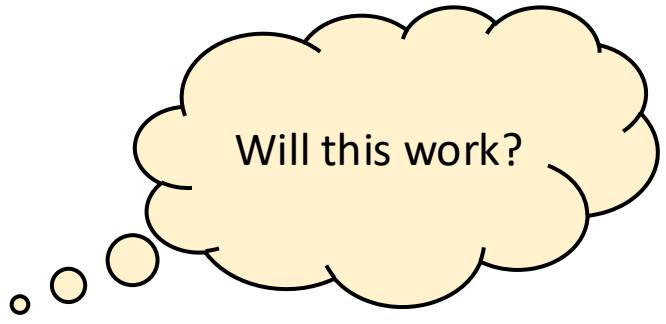
Skip one instruction → Skipping a precise number of instructions

Redundant Condition Checks

```
if (lc_state != kLcStateProd) {
   assert();
}
```

Redundant Condition Checks – First Attempt

```
if (lc_state != kLcStateProd) {
    assert();
}
assert(lc_state != kLcStateProd);
```



Smart compilers will perform dead code elimination.

Redundant Condition Checks - launder32

```
inline uint32_t launder32(uint32_t val) {
  asm volatile("" : "+r"(val));
  return val;
}
```

hardened.h

Redundant Condition Checks

```
if (launder32(lc_state) != LcStateProd) {
    assert();
}
 HARDENED_CHECK_NE(lc_state, LcStateProd);
```

A custom macro that
1) produces the `bne/beq` instruction directly with **inline assembly** and
2) adds some `unimp` padding afterwards.

Redundant Condition Checks

C	Assembly

`if (launder32(lc_state_check) != lc_state) {`
`  HARDENED_TRAP();`
`}`
`HARDENED_CHECK_EQ(lc_state_check, lc_state);`	

Redundant Condition Checks

C	Assembly

`if (launder32(lc_state_check) != lc_state) {`
`  HARDENED_TRAP();`
`}`
`HARDENED_CHECK_EQ(lc_state_check, lc_state);`
	`/proc/self/cwd/sw/device/silicon_creator/rom/rom.c:306`
	`  if (launder32(lc_state_check) != lc_state) {`
	`  91b0: lw  a2,-390(s1)`
	`  91b4: beq  a1,a2,91c4`
	
	`/proc/self/cwd/sw/device/silicon_creator/rom/rom.c:307`
	`  HARDENED_TRAP();`
	`  91b8: unimp`
	`  91ba: unimp`
	`  91bc: unimp`
	`  91be: unimp`

Why multiple instead of one unimp?

Redundant Condition Checks

C	Assembly

`if (launder32(lc_state_check) != lc_state) {`
`  HARDENED_TRAP();`
`}`
`HARDENED_CHECK_EQ(lc_state_check, lc_state);`
	`/proc/self/cwd/sw/device/silicon_creator/rom/rom.c:306`
	`  if (launder32(lc_state_check) != lc_state) {`
	`  91b0: lw  a2,-390(s1)`
	`  91b4: beq  a1,a2,91c4`
	
	`/proc/self/cwd/sw/device/silicon_creator/rom/rom.c:307`
	`  HARDENED_TRAP();`
	`  91b8: unimp`
	`  91ba: unimp`
	`  91bc: unimp`
	`  91be: unimp`
	
	`/proc/self/cwd/sw/device/silicon_creator/rom/rom.c:309`
	`  HARDENED_CHECK_EQ(lc_state_check, lc_state);`
	`  91c0: lw  a1,-390(s1)`
	`  91c4: beq  a0,a1,91d0`
	`  91c8: unimp`
	`  91ca: unimp`
	`  91cc: unimp`
	`  91ce: unimp`

Mitigation Design Considerations

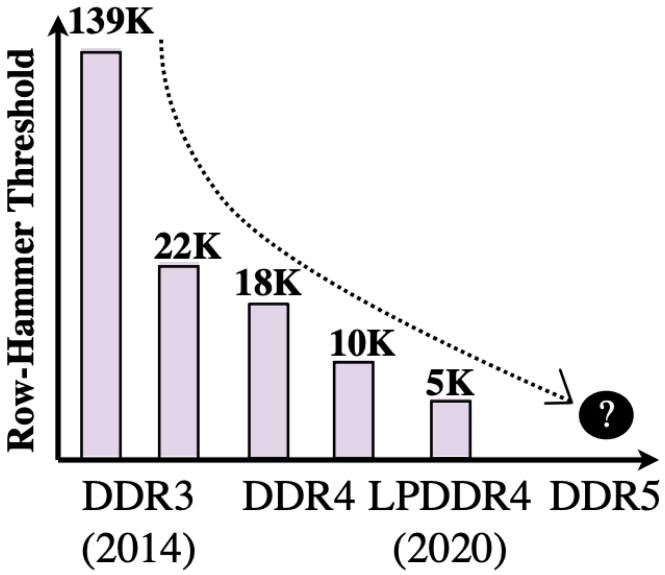
Cost?

Security?

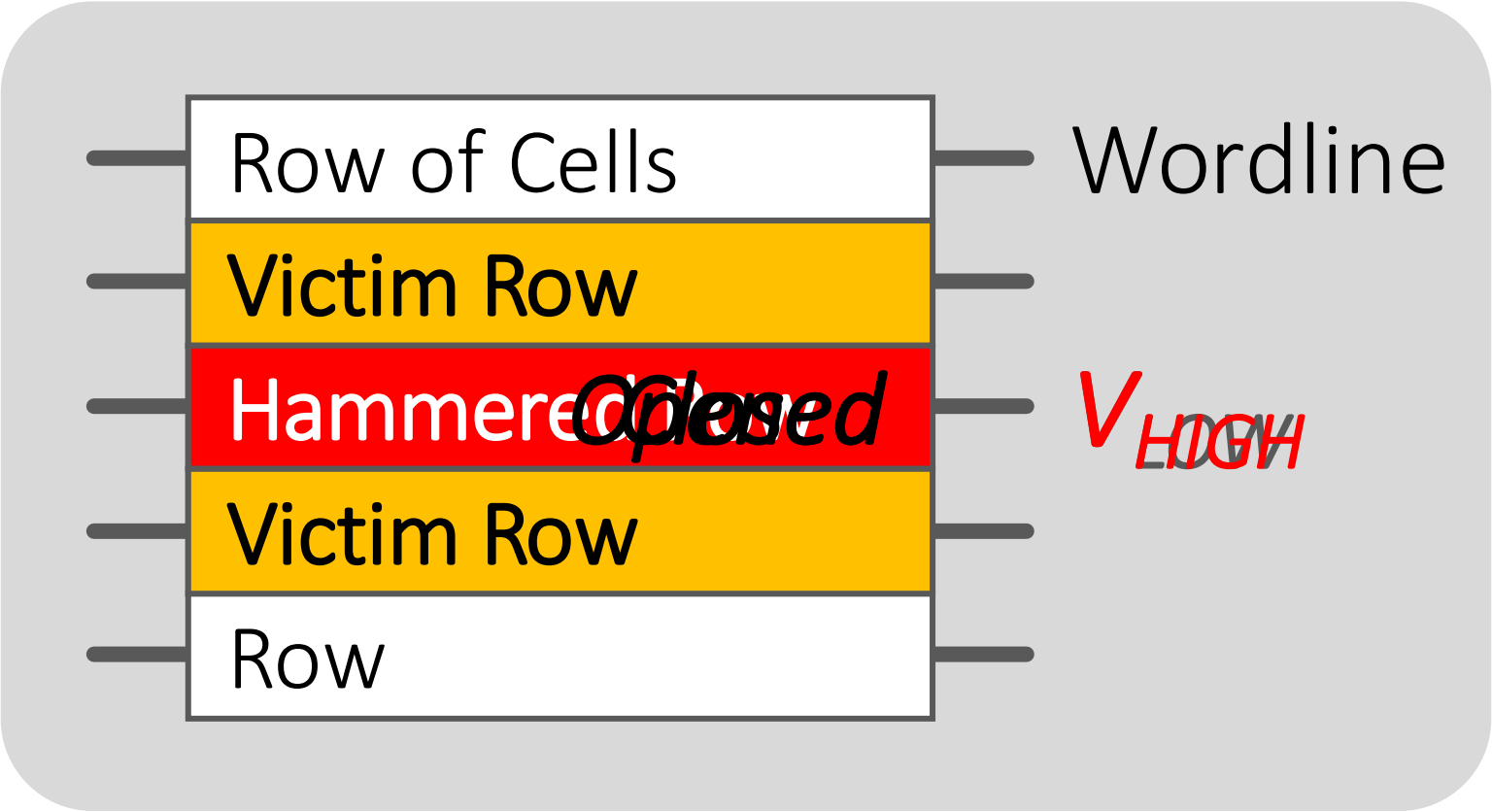
Usability?

Performance?

RowHammer Mitigations: A Numbers Game



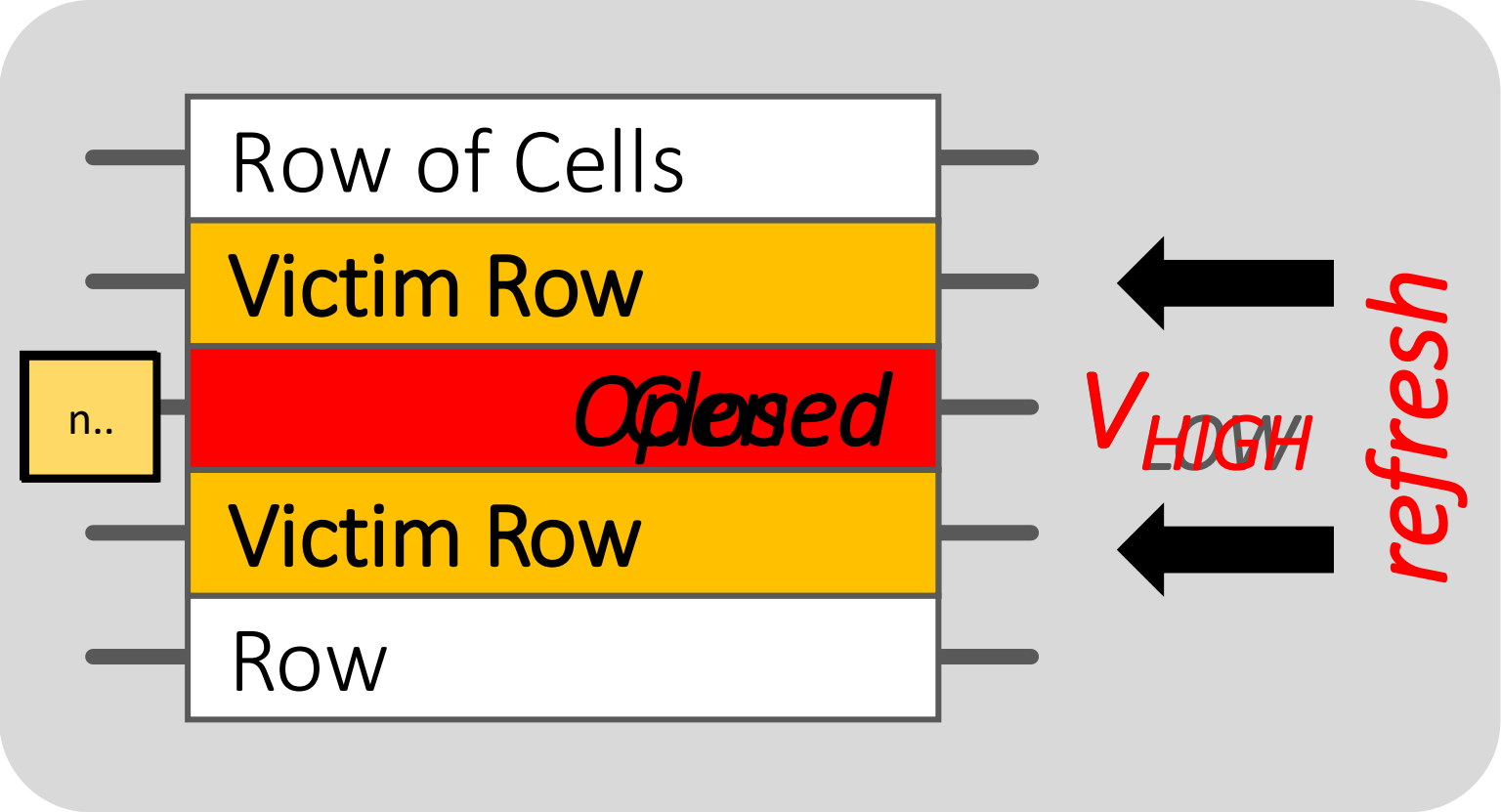
Recap RowHammer



Observation: Repeatedly accessing a row enough times **between refreshes** can cause disturbance errors in nearby rows

Targeted Row Refresh (TRR)

- **Assumption:**
  - If you refresh the victim row before its charge drops too far, no bit flip occurs.
- **Idea:**
  - Track frequent activations and triggers targeted refreshes of rows that are likely victims (rather than global refresh).



Targeted Row Refresh (TRR)

- Implemented as vendor-proprietary counters/heuristics in DRAM chips
- Problem 1: Limited tracking capacity
  - Most TRR implementations track only a **small number of hot rows**.
  - Why?

- How to bypass this mitigation?
  - Many-sided row hammer.
  - If k is 5, then activate sequence can be: R1, R2, R3, R4, R5, R6, R1, ……., R6….

Frigo et al. TRRespass: Exploiting the Many Sides of Target Row Refresh; SP'20

Targeted Row Refresh (TRR)

- Problem 2: Threshold/Window mismatch
  - TRR tracks activation counts within *64m* window
  - Why?

```
if activations(row) > 100K in 64ms:
        refresh neighbors
```

- In reality: Disturbance accumulates across multiple windows since charge recovery after refresh is not perfect
  - `[95K activations] → pause → [95K activations]`

Frigo et al. TRRespass: Exploiting the Many Sides of Target Row Refresh; SP'20

Mitigation Design Considerations

Cost?

Security?

Usability?

Performance?

Activation Trackers

- Goal: provably ensure below threshold

- Naïve: one counter per row
  - What is the problem?

- Do it smartly: using the Misra-Gries Algorithm
  - The Rowhammer tracking problem is very similar to the frequent elements problem
  - Given a stream of W items, the algorithm identifies **all the items** that appear more than T times, as long as

$$N_{entry} > W/T - 1$$

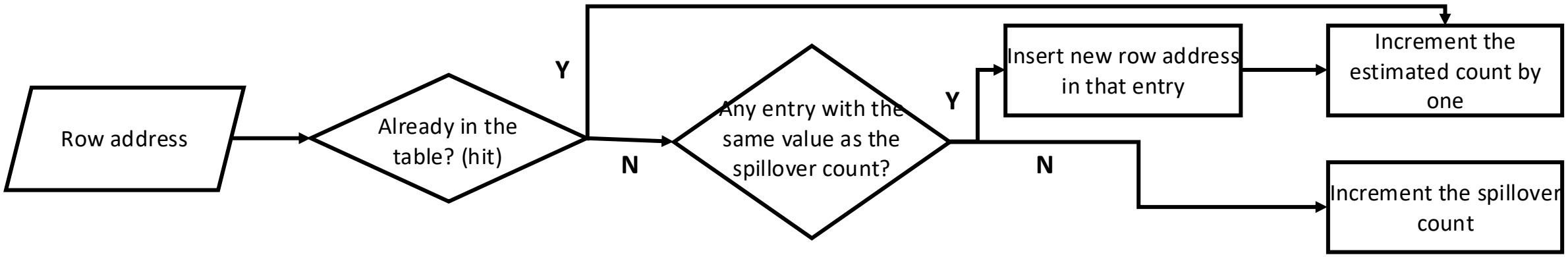
Row Address	Count
0x1010	5
0x2020	7
…	…

N_{entry}

Spillover Count	2

Graphene: Strong yet Lightweight Row Hammer Protection; Park et al; MICRO'20

Graphene Aggressor Tracking



Row Address	Count
0x1010	5
0x2020	7
0x3030	3

Spillover Count	2

Row Address	Count
0x1010	6
0x2020	7
0x3030	3

Spillover Count	2

Row Address	Count
0x1010	6
0x2020	7
0x3030	3

Spillover Count	3

Row Address	Count
0x1010	6
0x2020	7
0x5050	4

Spillover Count	3

Time

ACT (0x1010) | **ACT (0x4040)** | **ACT (0x5050)**

Graphene Analysis

$$N_{entry} > {}^{W}/_{T} - 1$$

- In the original paper (2020)
  - W Max number of ACTs in a refresh window: 1,360K
  - T Threshold for aggressor tracking: 12.5K (actual threshold = 32KB)
  - N_{entry} Number of table entries: 108
  - Each entry: 16 bits for row address; 15 bits for counting value up to T
  - Memory type: CAM (content addressable memory)

- In a recent paper, assuming 16GB memory (2022)
  - T Threshold for aggressor tracking: 250 (actual threshold = 500)
  - N_{entry} Number of table entries: 5440 (50x more)

Industry's Next Step: PRAC

- Introduced into the DDR5 spec update (April 2024).
- PRAC: **Per-Row** Activation Counting
  - Track *every* row's activations (not just top-K)
  - Signal controller when activation reaches threshold
    - Memory controller issues targeted row refresh
    - Throttling/backoff: temporally stall or slow requests to a given bank
    - Reschedule requests to the target row
- PRAC is a framework. Security depends on implementation quality.
- Tradeoff?

Mitigation Design Considerations

Cost?

Security?

Usability?

Performance?

Next:
Hardware Security Modules (HSM) / Root of Trust